WEEK 06 · MODEL & VALIDATION

收到資料
不等於可以接受

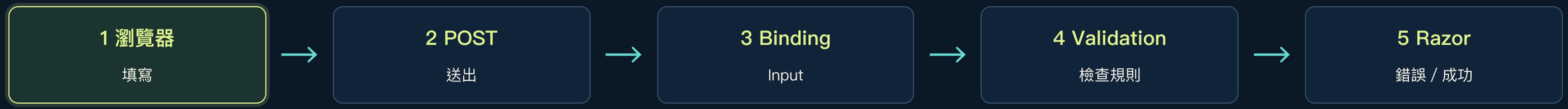
第五週已讓表單資料進入 PageModel；這週把 Input 看懂，再替資料加入清楚規則與可重現測試。

下載學生任務包 ↓　下載 PDF ↓



RECONNECT · 先找回資料路線

驗證不是另一套魔法，它插在收到與成功之間。



下一站　從頭看

先確認第五週的表單仍能把資料送到 Input。

MISSION MAP

三節課，把 Model 從名詞變成可操作的資料結構。

1

看懂 Input Model

從一個屬性開始，理解型別、物件與欄位對應。

2

加入驗證閘門

DataAnnotations 定義規則，ModelState 決定能否成功。

3

用測試證明

走空白、錯誤格式、合法資料三條路，再做同儕驗收。

MODEL 01 · DATA SHAPE

Model 先不用想得很抽象：它描述這筆資料有哪些欄位。

不是畫面

HTML 決定使用者看到與操作什麼。

不是資料庫

物件現在只存在記憶體，重新啟動不會留下。

是一個結構

屬性名稱與型別，說明這筆輸入長什麼樣子。

> 本週的 Model 指 `RegistrationInput`；PageModel 指整個 Razor Page 的後端處理類別。

MODEL 02 · ONE PROPERTY

一個屬性，就是物件裡一個有名字的位置。

先只看顯示名稱，不一次吞下整個 class。

```
public class RegistrationInput
{{
  public string DisplayName {{ get; set; }} = string.Empty;
}}
```



MODEL 03 · TYPE

型別先回答：這個位置可以放什麼？

`string`

文字：名稱、Email、場次、備註。

`int`

整數：未來若加入報名人數，可用數值規則。

`bool`

是 / 否：例如是否同意條款；不適合拿來存名稱。

型別不是驗證全部。`string` 能裝下 `abc`，但那不代表它是合法 Email。

MODEL 04 · INPUT MODEL

四個欄位，不要散成四條線；用一個 Input 物件收在一起。

```
public class RegistrationInput
{
    public string DisplayName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Session { get; set; } = string.Empty;
    public string? Note { get; set; }
}
```

必填文字

string＋空字串起點

選填文字

string? 可沒有值

物件

四個屬性收成一組

? 表示允許沒有值；它不會自動限制長度或 Email 格式。

MODEL 05 · MAPPING

asp-for 寫的路徑，
就是 Binding 要找的目的地。

Razor 欄位

```
<input asp-for="Input.DisplayName">
```

C# 目的地

```
public RegistrationInput Input {{ get; set; }}
public string DisplayName {{ get; set; }}
```

> Input.DisplayName 先找 Input，再找裡面的 DisplayName；任何一段拼錯都會斷線。

讓學生用手指出路徑的兩段

暫停講解 · 學生實作

PAUSE & PRACTICE 00

開兩個檔案，沿欄位找到 C# 目的地。

15 分鐘

查看預期成品 Demo ↗

照做步驟

1. 開啟 `Pages/Index.cshtml.cs`，找到 `IndexModel`。
2. 在同一檔案、`IndexModel` 外找到 `RegistrationInput` 類別；再到 `IndexModel` 內找到 `Input` 屬性。
3. 畫出 `IndexModel → Input → RegistrationInput` 四個欄位。
4. 在每個欄位旁寫下型別：`string` 或 `string?`。
5. 再開 `Index.cshtml`，選一個 `asp-for` 畫線連回同名屬性。

逐項自驗

- `Input.DisplayName` 能分成 Input 與 DisplayName 兩段。
- 能指出 `RegistrationInput` 是同檔案的另一個類別，`IndexModel.Input` 指向它。
- 能說明 `string?` 可沒有值，但不代表已驗證格式。
- 拼錯任一段時，能指出 Binding 會找不到目的地。

卡住先查：沒有 Input 時，先回看第五週完成檔，不要自行發明類別名稱。

第 8 分鐘：先完成 DisplayName、Email 與一條 asp-for 連線。

快學生：解釋為何 Input Model 不是資料庫資料表。

☐ 我已開兩個檔案，並從 asp-for 指到正確 C# 屬性。

時間到：抽一位學生沿 asp-for 指到 C# 屬性

CHECKPOINT 01 · 學生操作

不要背答案：開檔案、指出位置、再說三句話。

1

先找 PageModel

開 `Index.cshtml.cs`，用手指出 `IndexModel`：它處理這個 Razor Page 的請求與狀態。

2

再找 Input Model

指出 `RegistrationInput`：它把一次表單輸入收成有結構的物件。

3

最後走 Binding

開 `Index.cshtml` 找 `asp-for`，沿名稱連到 Input 的同名屬性。

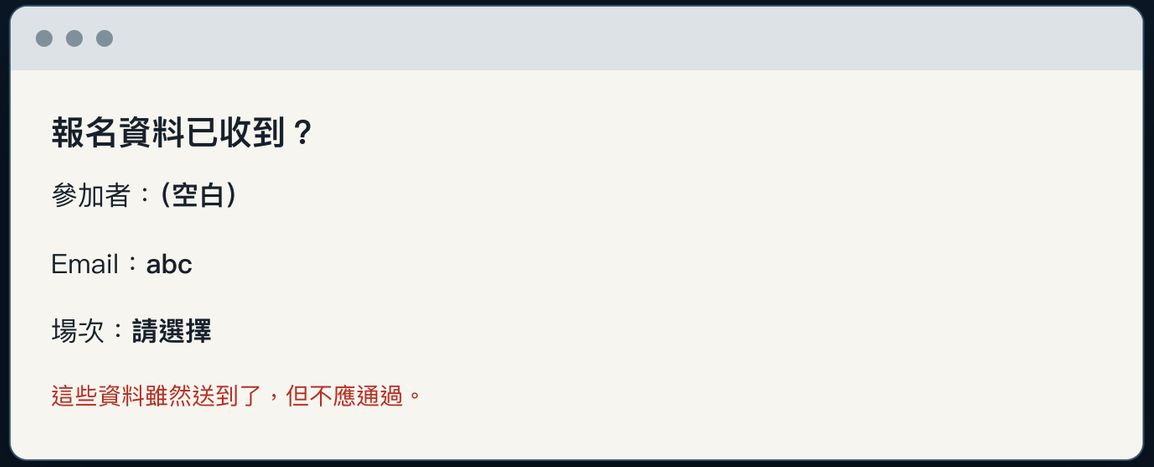
自驗句型：「畫面上的____，經 Binding 放進 `Input.____`；PageModel 再處理這次 POST。」三格都指得出實際程式位置才過關。

找不到時：在 `Index.cshtml.cs` 搜尋 `class RegistrationInput` / `Input`；在 `Index.cshtml` 搜尋 `asp-for=`。名稱不同只修不一致的一段，Build 後再沿線一次。

VALIDATION 01 · RECEIVED ≠ VALID

後端收到資料，
不代表資料合理。

空白名字、錯誤 Email、未選場次都能被傳送。驗證的工作，是在成功之前設一道明確閘門。



VALIDATION 02 · RULES BESIDE DATA

每個驗證標記，只回答一個問題。

```
[Required(ErrorMessage = "請輸入顯示名稱")]
[StringLength(20, ErrorMessage = "最多 20 字")]
[Display(Name = "顯示名稱")]
public string DisplayName { get; set; }
```

Required

可以空白嗎？

StringLength

最多可以多長？

Display

畫面上要叫什麼名字？

一條規則，一個可測試問題

VALIDATION 03 · ONE RULE

不要背整串；
逐條問它在防什麼。

[Required]

有沒有填？空白不能通過。

[EmailAddress]

有填時，格式像不像 Email？

[StringLength(20)]

文字長度是否超過上限？

[Display] 改善顯示名稱，本身不是「擋資料」的驗證規則。

VALIDATION 04 · VALIDATION GATE

先問 ModelState.IsValid，通過才打開成功區。

```
public void OnPost()
{
    if (!ModelState.IsValid)
    {
        return;
    }

    Submitted = true;
}
```

規則未通過

留在表單，顯示錯誤

全部通過

顯示成功摘要

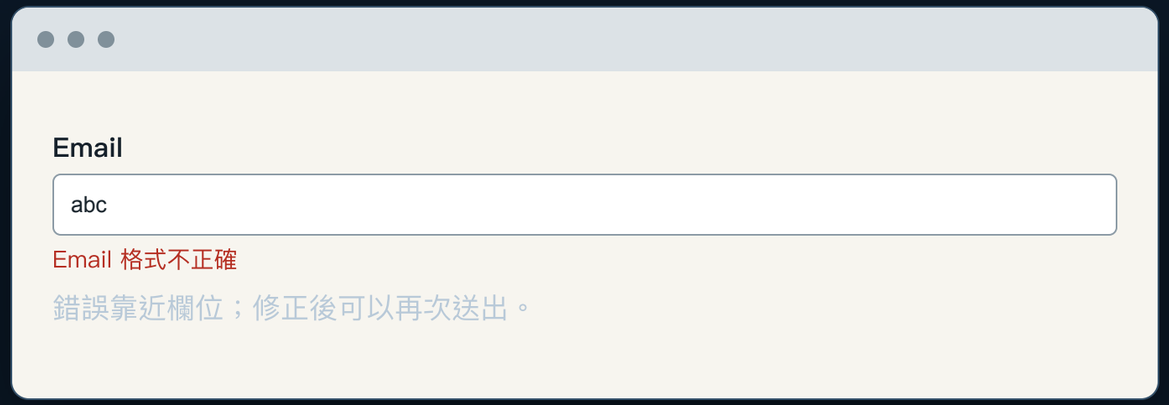
如果先設定 Submitted，再檢查 ModelState，空白資料也可能出現成功訊息。

成功訊息必須在驗證之後

VALIDATION 05 · SHOW THE REPAIR PATH

驗證不是只說「失敗」，要讓使用者知道哪裡需要修。

```
<label asp-for="Input.Email"></label>
<input asp-for="Input.Email"
       class="form-control">
<span asp-validation-for="Input.Email"
      class="text-danger"></span>
```



錯誤訊息本身也是介面設計

暫停講解 · 學生實作

PAUSE & PRACTICE 01

依檔案順序，讓錯的停下、對的通過。

20 分鐘

查看預期成品 Demo ↗

照做步驟

1. 在 `Index.cshtml.cs` 頂端加入 `using System.ComponentModel.DataAnnotations;`。
2. 名稱加 `[Required]`、`[StringLength(20)]`；Email 加 `[Required]`、`[EmailAddress]`；Session 也加 **`[Required]`**。
3. 在 `OnPost` 最前面加入完整閘門：`if (!ModelState.IsValid) { return; }`。
4. 到 `Index.cshtml`，在名稱、Email、Session 旁加入同路徑的 `asp-validation-for`。
5. 啟動 localhost，依序送出空白、`abc`、未選場次、合法假資料。

每一步看什麼

- Build 成功，三個必填屬性上方的標記沒有紅線。
- 空白、錯誤 Email、未選場次：留在表單、顯示對應錯誤、沒有成功摘要。
- 修正欄位再送：錯誤消失，成功摘要與輸入一致。
- 若錯誤沒出現，依序查屬性標記、ModelState 閘門、同路徑 validation-for。

卡住先查：規則放在 Input 屬性上；Session 的 `[Required]` 不可省略。

第 10 分鐘：名稱、Email、Session 都必填，再接錯誤訊息。

快學生：測名稱 20 / 21 字，不增加欄位。

- [ ] 我已在 localhost 完成空白、錯誤格式、正確資料三條測試。

時間到：兩人互換測試，不直接替對方改程式

CHECKPOINT 02 · 排錯操作

驗證沒反應時，照四站逐一打開檢查。

1 屬性
開 cshtml.cs：標記在正確屬性上？

→

2 Handler
OnPost 先擋無效 ModelState？

→

3 Razor
欄位旁有同路徑 validation-for？

→

4 測試
重新送出失敗＋成功資料

> 一次只改一站：先記錄「輸入、預期、實際」，找到第一個不一致的位置，修改後重新 Build 並重跑同一組資料。

TEST 01 · TEST BEHAVIOR

只測成功一次，不能證明驗證有效。

測試路線	輸入	應該看到	操作
空白	所有必填欄空白	必填錯誤；沒有成功摘要	模擬
錯誤格式	Email = abc	Email 格式錯誤	模擬
正確	課堂假資料	成功摘要與輸入一致	模擬

選一條路，觀察預期行為。之後要在自己的 localhost 重做。

TEST 02 · BOUNDARY

上限 20 字，至少要測 20 字成功、21 字失敗。

19

上限內

應成功，但還沒碰到真正邊界。

20

剛好上限

應成功，證明沒有少允許一個字。

21

超過一個

應失敗，證明後端規則真的生效。

測規則的邊界，不只測一個漂亮範例

TEST 03 · AI DEBUG CONTRACT

不要請 AI 重寫整頁，給它一個可重現的小問題。

✕ 太模糊

```
表單壞了，幫我全部修好。
```

AI 不知道輸入、操作、預期與實際結果。

✓ 可驗證

```
Razor Pages 表單使用 POST。
輸入 Email=abc 時仍顯示成功。
以下只貼 OnPost 與 Email 屬性。
請先指出原因，再給最小修改與三組測試。
```

讀 理解建議 → **比** 查看差異 → **跑** 啟動專案 → **測** 三條路 → **追問** 描述現象

暫停講解 · 學生實作

PAUSE & PRACTICE 02

每組資料都先寫預期，再按送出並記結果。

20 分鐘

查看預期成品 Demo ↗

照做步驟

1. 啟動 localhost，清空所有必填欄；先寫「應顯示必填錯誤」，再送出。
2. 填合法名稱，Email 輸入 abc；先寫「應顯示格式錯誤」，再送出。
3. 改成課堂合法假資料；先寫「應顯示成功摘要」，再送出。
4. 名稱分別貼入 20 字與 21 字，再各送出一次。
5. 每次記下輸入、預期、實際；不一致就回第17頁四站排查。

要留下的證據

- 空白：欄位旁有錯誤，沒有成功摘要。
- abc：Email 旁有格式錯誤。
- 合法資料：摘要與輸入逐欄一致。
- 20 字成功、21 字失敗；能說明這是邊界測試。

卡住先查：失敗畫面也要截到輸入與錯誤，不只截紅字。

第 10 分鐘：至少留下空白、abc、合法三路紀錄。

快學生：交換電腦重跑 20 / 21 字，不看對方程式。

- [ ] 我已留下每條測試的輸入、預期與實際結果。

時間到：留下三條可重現的測試證據

PAIR TEST · 學生操作

交換電腦後，測試者照表操作，不先看程式。

1

選一條路

先選空白、錯誤 Email 或合法資料，寫下完整輸入與預期。

2

實際操作

在同學的 localhost 輸入資料、按送出；不要替對方修改程式。

3

交付回報

記下實際畫面，指出預期與實際第一個不同的位置，再交回原作者。

回報句型：「我在____頁輸入____，按下____後，預期____，實際____；可先檢查第____站。」原作者重現後才修改。

只用課堂假資料；不要輸入真實 Email、電話、密碼、連線字串或 API key。

好回報能讓下一個修改更小、更準

FINAL BOSS · REPAIR WITH EVIDENCE

不是改一個字就過關，要重現、修正、測試、說明。

基礎救援
一個問題，有明確修改範圍

標準任務
自己定位，測成功與失敗

進階委託
加入小需求與邊界測試

抽情境任務

基礎救援　8–12 分鐘

幽靈報名成功

情境：所有欄位空白，畫面卻顯示報名成功。

任務：調整 OnPost，讓錯誤資料停在表單。

限制：保留 DataAnnotations；必須使用 ModelState.IsValid。

必測：空白失敗；合法資料成功。

口頭說明：為什麼要先檢查再設定 Submitted？

☐ 老師已看過我的重現、修改、測試與說明，確認通過。

同級抽題，避免運氣造成難度失衡

MISSION COMPLETE · WEEK 06

你現在不只會接資料，還能解釋、擋錯、測試。

Model 說得清楚

分得出 PageModel、Input Model 與資料庫。

規則有閘門

DataAnnotations 與 ModelState 各負責一件事。

測試有證據

空白、錯誤格式、合法資料都能重現。

☐ 我能解釋 Input Model ☐ 驗證規則與閘門完成 ☐ 三路測試有證據 ☐ 老師已驗收 Boss

仍未寫入資料庫；下一步才是多筆清單與持久保存。